IBM HashiCorp

VAULT ON KUBERNETES

Same Vault, same app, two operators

Vault Secrets Operator vs External Secrets Operator

A side-by-side lab: Vault Enterprise 2.1 · VSO 1.6 · ESO · Postgres · kind

THE QUESTION

Getting Vault secrets into a pod is easy. Keeping them fresh, short-lived and cleaned up is the real job

Freshness

When a secret rotates, how long until every workload is running on the new value?

Lifecycle

Who renews dynamic credentials, and who revokes them when a workload goes away?

Operations

How many components does it take, and who supports them when they break?

Both operators sync Vault data into Kubernetes Secrets. The differences are in these three areas.

AT A GLANCE

Two operators, different design goals

	VAULT SECRETS OPERATOR	EXTERNAL SECRETS OPERATOR
Built for	Vault, in depth	Many backends through one API
Maintained by	HashiCorp (IBM)	Open-source community
Support	Covered by Vault Enterprise support	Community; third-party options
How it syncs	Vault events, with a refresh fallback	Polling on a refresh interval
Dynamic secrets	Lease-aware: renew, rotate, revoke	Generator mints a new secret each refresh
App restart	Built in (`rolloutRestartTargets`)	Add-on (e.g. Stakater Reloader)
Write back to a store	No	Yes (`PushSecret`)

A fair fight: only the operator changes

Vault Enterprise · namespace `vso-demo`

`kv/` `database/` `pki/` `vso-transit/` `auth/kubernetes`

app-vso

`VaultStaticSecret` · `VaultDynamicSecret` · `VaultPKISecret`

Policy: read + **subscribe** (for events)

app-eso

`SecretStore` · 2 generators · 3 `ExternalSecrets`

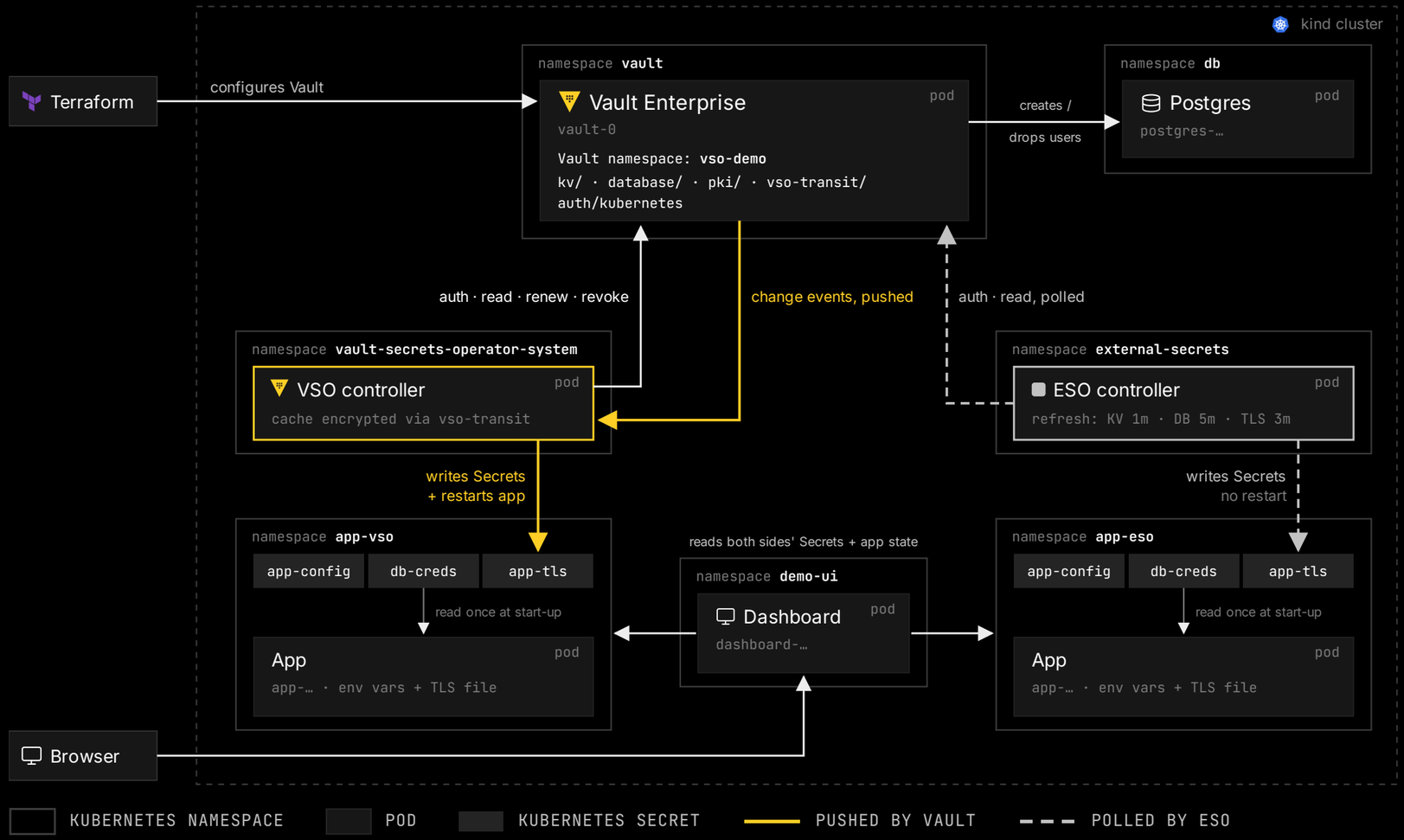
Refresh: KV 1m (default 1h) · DB 5m of a 10m TTL · TLS 3m of 5m

Identical app on both sides: reads its secrets once at start-up (env vars + a TLS file), like most real applications. Same Postgres, same auth method, same service account name.

Dashboard: localhost:8080 · Vault UI: localhost:8200

Config: terraform/ · k8s/app-vso · k8s/app-eso

How the lab fits together



SCENARIO 1 · FIRST SYNC OF A KV SECRET

Getting a KV secret into the cluster: both do the job

VSO `VaultStaticSecret`

- Reads `kv/app/config` from Vault
- Creates the Kubernetes Secret `app-config`
- The app starts with the value

IN SYNC

ESO `ExternalSecret`

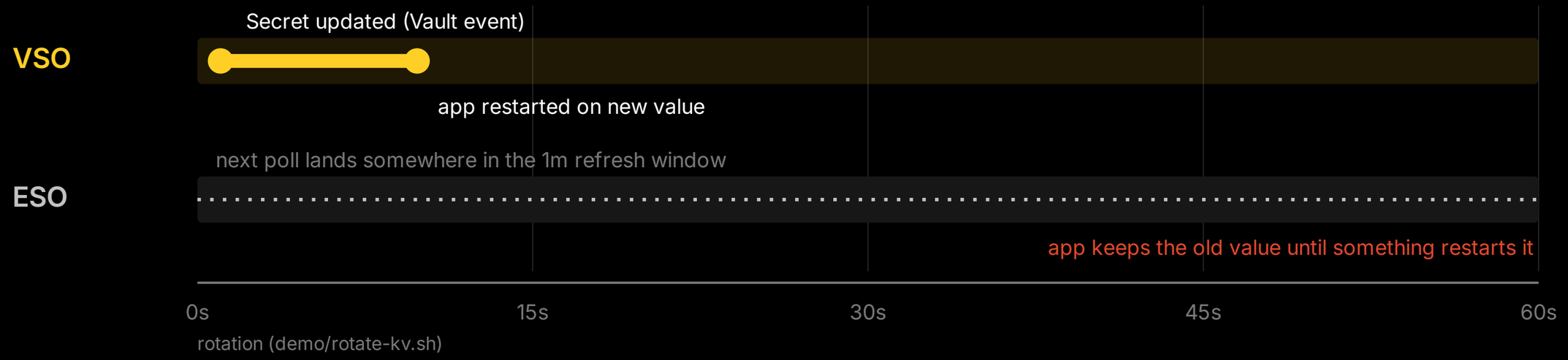
- Reads `kv/app/config` from Vault
- Creates the Kubernetes Secret `app-config`
- The app starts with the value

IN SYNC

Same result on day one. The difference appears when the secret changes in Vault, which is next.

SCENARIO 2 · ROTATION

Events vs polling: how fast does a rotation land?



Polling also means constant reads and audit noise on Vault, even when nothing changes.

Live `demo/rotate-kv.sh`

SCENARIO 3 · RELOAD

The Secret changing isn't the finish line. The process has to pick it up

VSO

- `rolloutRestartTargets` on each secret resource
- Rolls the Deployment after every sync, rotation or renewal

BUILT IN

ESO

- No built-in restart
- Add Stakater Reloader and annotate each workload
- One more component to accredit, patch and trust

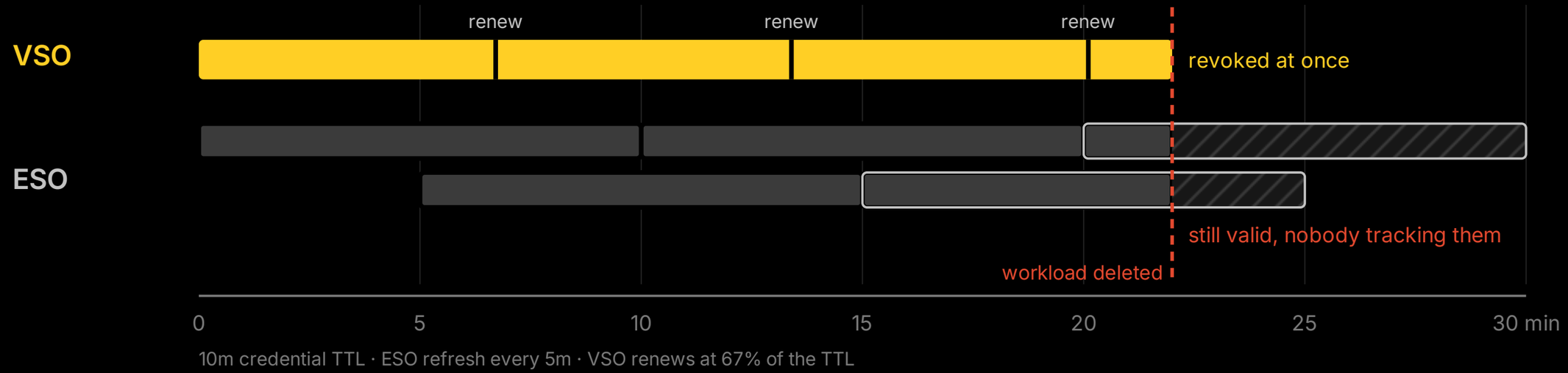
ADD-ON REQUIRED

Optional: show ESO with Reloader to make the comparison fair.

Live `demo/eso-reloader.sh`

SCENARIO 4 · DYNAMIC DATABASE CREDENTIALS

Who owns the lease?



10m credential TTL · ESO refresh every 5m · VSO renews at 67% of the TTL

VSO: one credential, renewed, revoked on delete. ESO: a new credential each refresh, old ones left to expire.

Live demo/leases.sh

SCENARIO 4 · DELETE THE WORKLOAD

When a workload is decommissioned, how long do its database credentials stay live?

Seconds

VSO

VSO revokes the lease when the VaultDynamicSecret is deleted. The Postgres user is dropped.

Up to the TTL

ESO

ESO leaves its credentials to expire on their own. Nothing in Kubernetes still knows they exist.

Put it back afterwards: demo/restore.sh

Live `demo/delete-db.sh vso · eso`

SCENARIO 5 · SHORT-LIVED TLS

Short-lived certificates only work when renewal and reload are automatic

VSO `VaultPKISecret`

- Renews 30 seconds before a 5-minute certificate expires
- Rolls the app onto the new certificate
- Revokes the old one

IN SYNC

ESO `generator → pki/issue`

- New certificate in the Secret every 3 minutes
- The app keeps serving the one it loaded at start-up
- Old certificates aren't revoked

APP SERVING AN EXPIRED CERT

Watch row 3 of the dashboard. The serial changes on its own on the VSO side.

EVEN WITH RELOADER

Reloader keeps ESO's apps current. The cost just moves

	VAULT SECRETS OPERATOR	ESO + STAKATER RELOADER
DB credentials issued	2 per workload per hour (renewed in place)	12 per workload per hour (new every refresh)
Live at any moment	1 per workload	2–3 per workload, none renewed
Workload deleted	Revoked within seconds	Stay valid until their TTL runs out
KV reads against Vault	On change (events), plus an hourly fallback	Every refresh: 60 per secret per hour
Vault shortens a TTL	Adapts to the lease and certificate expiry	Expired until every manifest is re-tuned
Components to accredit	One, under Vault Enterprise support	Two community projects

Per workload, lab settings: DB TTL 10m, max 30m; ESO refresh 5m (DB), 1m (KV)

Live `demo/tighten-ttl.sh`

SUMMARY

VSO for Vault. ESO for what isn't in Vault

Choose VSO when

- Vault is the system of record
- You use dynamic credentials or PKI
- Rotation needs to land in seconds
- You want it under the same support contract as Vault Enterprise

Where ESO still fits

- Secrets that live in other stores, such as AWS Secrets Manager or Azure Key Vault
- Pushing secrets from Kubernetes back to a store (PushSecret)

Anything that lives in Vault: VSO.